Chapter 3. HTML 시맨틱과 Tailwind CSS

> **미션**: Copilot과 함께 블로그 레이아웃을 시맨틱 태그로 마크업하고 Tailwind CSS로 스타일링한다

학습목표

1. HTML5 시맨틱 태그의 의미를 이해하고 올바르게 사용할 수 있다
2. Tailwind CSS의 유틸리티 클래스를 읽고 해석할 수 있다
3. Flexbox와 Grid로 레이아웃을 구성하고 반응형을 적용할 수 있다
4. AI가 생성한 HTML/CSS 코드에서 흔한 실수를 발견할 수 있다

---

0. HTML과 CSS 기초

이 장은 "바로 예쁘게 만들기" 전에, HTML과 CSS가 각각 무슨 역할인지 먼저 확인하고 시작한다.

먼저 큰 그림 보기

학생들이 자주 헷갈리는 부분은 "지금 내가 배우는 게 정확히 무엇인가?"이다.

아주 간단히 정리하면:

1. HTML: 웹페이지의 구조를 만든다
2. CSS: 웹페이지의 모양을 꾸민다
3. JavaScript: 웹페이지를 동작하게 만든다
4. React: JavaScript로 화면을 더 체계적으로 만들기 위한 도구
5. Next.js: React 프로젝트를 더 편하게 만들기 위한 프레임워크

즉, 지금 우리는 "HTML/CSS/JavaScript 위에 React와 Next.js를 올려서" 웹페이지를 만드는 중이다.

비유하면:

- HTML: 뼈대
- CSS: 디자인
- JavaScript: 움직임과 동작
- React: 화면을 부품처럼 나눠 만드는 방식
- Next.js: React 앱을 실제 서비스 형태로 구성하는 작업 도구

아주 짧은 흐름 정리

웹의 시작점은 HTML이었다. 처음에는 문서를 보여주는 것이 핵심이었다.

그다음 CSS가 붙으면서 웹페이지의 색, 글꼴, 여백, 배치 같은 디자인을 조절할 수 있게 되었다.

이후 JavaScript가 들어오면서 버튼 클릭, 메뉴 열기, 입력 처리처럼 "사용자와 상호작용하는 웹"이 가능해졌다.

하지만 JavaScript만으로 큰 화면을 관리하기는 점점 어려워졌다. 그래서 React 같은 라이브러리가 등장했고, 화면을 컴포넌트 단위로 나누어 만들 수 있게 되었다.

Next.js는 React를 기반으로 만들어진 프레임워크이다. 폴더 구조, 페이지 구성, 배포, 서버 기능 등을 더 쉽게 다룰 수 있게 도와준다.

이번 장에서는 이 전체 흐름 중에서 가장 바닥에 있는 HTML 구조와 CSS 스타일링에 집중한다.

CSS도 발전해 왔다

CSS도 처음부터 Tailwind 같은 형태였던 것은 아니다.

아주 단순하게 보면 흐름은 이렇다.

1. 기본 CSS: `style.css` 파일에 직접 작성
2. 더 큰 프로젝트용 CSS: Sass/SCSS처럼 코드를 더 편하게 관리
3. 컴포넌트 중심 CSS: 이미 만들어 둔 버튼, 카드 같은 완성품을 가져다 쓰는 방식
4. 유틸리티 중심 CSS: 색상, 여백, 글자 크기 같은 작은 부품을 조합해 만드는 방식

즉, Tailwind는 CSS를 대체하는 기술이 아니라, CSS를 더 빠르고 일관성 있게 쓰기 위해 발전한 방식 중 하나이다.

이번 장에서는 CSS의 모든 역사를 배우지는 않는다. 대신 "HTML로 구조를 만들고, Tailwind CSS로 모양을 조절한다"는 실습 흐름에 집중한다.

핵심 한 줄:

- HTML은 구조를 만든다
- CSS는 모양을 바꾼다

예를 들어 블로그 카드 1개를 만든다고 하자.

```
<article>
  <h3>React 19 새 기능 정리</h3>
  <p>React 19에서 달라진 핵심 기능을 간단히 정리했다.</p>
</article>
```

위 코드는 "무엇이 들어 있는지"를 보여주는 구조이다. 아직 글자 크기, 색, 여백은 거의 정해지지 않았다.

여기에 CSS를 더하면 모양이 바뀐다.

```
article {
  padding: 24px;
  background-color: white;
}

h3 {
  font-size: 20px;
}
```

즉:

- HTML: 제목, 본문, 메뉴, 푸터 같은 내용을 배치한다
- CSS: 색, 글자 크기, 여백, 정렬 같은 모양을 조절한다

HTML 기본 문법

HTML 태그는 보통 여는 태그와 닫는 태그로 이루어진다.

```
<p>안녕하세요</p>
```

구조를 보면:

- `<p>`: 시작
- 안녕하세요: 내용
- `</p>`: 끝

태그 안에는 속성을 넣을 수 있다.

```
<a href="/about">   소개 페이지   </a>
```

- `a`: 링크 태그
- `href`: 어디로 이동할지 알려주는 속성

JSX에서 특히 주의할 것

우리는 순수 HTML 파일이 아니라 `app/page.tsx`를 수정한다. 그래서 HTML과 비슷해 보여도 JSX 규칙을 따라야 한다.

JSX가 무엇인가

JSX는 JavaScript 안에서 HTML처럼 보이는 코드를 쓰는 문법이다.

예를 들어 아래 코드는 HTML처럼 보이지만, 실제로는 React 컴포넌트 안에 작성하는 JSX이다.

```
export default function Home() {
  return <h1>안녕하세요</h1>;
}
```

학생 입장에서는 이렇게 이해하면 충분하다:

- HTML처럼 생겼다
- 하지만 `tsx` 파일 안에서 쓴다
- 그래서 완전히 똑같지는 않고 JSX 규칙을 따른다

즉, 우리는 "HTML 비슷하게 생긴 React 코드"를 쓰고 있는 것이다.

자주 틀리는 것:

```
{/* ❌ HTML 방식 */}
<div class="card">
  <label for="title">제목</label>
</div>

{/* ☑ JSX 방식 */}
<div className="card">
  <label htmlFor="title">제목</label>
</div>
```

반드시 기억:

- `class`가 아니라 `className`
- `for`가 아니라 `htmlFor`

CSS를 쓰는 3가지 방식

웹에서 스타일을 주는 방식은 여러 가지가 있다.

1. CSS 파일에 직접 작성
2. 태그에 inline style 작성
3. Tailwind CSS 클래스 사용

이 장에서는 3번을 사용한다.

```
<article className="p-6 bg-white rounded-lg shadow">
  <h3 className="text-lg font-bold">React 19 새 기능 정리</h3>
</article>
```

이 코드는 "Tailwind 클래스로 CSS를 적용한 것"이다. 즉, Tailwind는 CSS를 안 쓰는 것이 아니라, CSS를 클래스 형태로 빠르게 쓰는 방법이다.

먼저 확인하고 갈 최소 태그

이번 장에서 자주 보는 태그는 아래 정도이다.

태그	의미
`<h1>`	페이지의 가장 큰 제목
`<h2>`	섹션 제목
`<p>`	문단
`<a>`	링크
`<header>`	페이지 위쪽 영역
`<nav>`	메뉴

태그	의미
`<main>`	핵심 본문
`<article>`	독립된 글/카드
`<footer>`	페이지 아래쪽 영역

빠른 점검

아래 코드를 보고 HTML과 CSS 역할을 구분해본다.

```
<article className="p-4 bg-white">
  <h3 className="text-lg font-bold">제목</h3>
  <p className="text-gray-600">내용</p>
</article>
```

- HTML 역할: `article`, `h3`, `p`로 구조를 나눈다
- CSS 역할: `p-4`, `bg-white`, `text-lg`, `font-bold`로 모양을 바꾼다

이 구분이 되면 다음 실습으로 넘어갈 준비가 된 것이다.

실습 전 준비

이 장은 `app/page.tsx`를 수정하여 블로그 메인 페이지를 만드는 실습이다.

실습 시작 전 먼저 확인:

- ☐ 프로젝트 폴더가 VS Code에서 열려 있는가?
- ☐ 터미널에서 `npm run dev`를 실행했는가?
- ☐ 브라우저에서 `http://localhost:3000`이 열리는가?
- ☐ 수정 대상 파일이 `app/page.tsx`라는 것을 확인했는가?

문제가 생기면 먼저 확인:

1. 개발 서버가 꺼져 있지 않은가?
2. `app/page.tsx`를 수정하고 있는가?
3. 저장 후 브라우저를 새로고침했는가?

이번 실습의 완성 목표

최종 결과는 아래와 같은 블로그 홈 화면이다.

```
┌──────────────────────────────────────┐
│ 블로그 제목              메뉴 링크들    │
├──────────────────────────────────────┤
│ 최신 게시글                            │
│ ┌──────────────┐  ┌──────────────┐    │
```

```
│ │ 게시글 카드 │  │ 게시글 카드 │        │
│ └────────────┘  └────────────┘        │
│ ┌────────────┐                         │
│ │ 게시글 카드 │                         │
│ └────────────┘                         │
├────────────────────────────────────────┤
│ footer                                 │
└────────────────────────────────────────┘
```

게시글 카드 1개에는 최소한 아래 정보가 들어간다.

- 제목
- 내용 미리보기
- 작성자
- 날짜

예시 게시글 데이터:

```
제목: React 19 새 기능 정리
미리보기: React 19에서 달라진 핵심 기능을 간단히 정리했다.
작성자: 김코딩
날짜: 2026-03-30
```

3.1 HTML 시맨틱 구조

시맨틱 태그란

`<div>`는 "아무 의미 없는 상자"이다. **시맨틱 태그**는 "이 영역이 무엇인지" 알려주는 이름표가 붙은 상자이다.

시맨틱 태그를 써야 하는 이유:

1. **검색 엔진**이 페이지 구조를 이해한다 (SEO)
2. **스크린 리더**가 시각 장애 사용자에게 구조를 안내한다 (접근성)
3. **AI(Copilot)**가 영역의 목적을 파악하여 정확한 코드를 생성한다

주요 시맨틱 태그

태그	역할	블로그 예시
`<header>`	머리말 영역	사이트 제목, 로고
`<nav>`	내비게이션	메뉴 링크
`<main>`	핵심 콘텐츠 (1페이지 1개)	게시글 목록
`<article>`	독립적 콘텐츠	게시글 1개
`<aside>`	보조 정보	사이드바, 카테고리
`<footer>`	바닥글 영역	저작권, 연락처

블로그 레이아웃 구조:

```
┌──────────────────────────────┐
│          <header>            │
│  사이트 제목 + <nav> 메뉴    │
├──────────────────────────────┤
│ <main>                       │
│  <article> <article> ...     │
├──────────────────────────────┤
│          <footer>            │
└──────────────────────────────┘
```

heading 계층

heading 태그(`<h1>`~`<h6>`)는 반드시 순서대로 사용한다. 한 페이지에 `<h1>`은 1개만.

```html
<!-- ☑ 올바른 -->
<h1>블로그</h1>
<h2>최신 게시글</h2>
<h3>React 19 새 기능</h3>

<!-- ✕ h2를 건너뛰면 안 됨 -->
<h1>블로그</h1>
<h3>최신 게시글</h3>
```

> **AI 주의**: Copilot은 heading 계층을 무시하고 시각적 크기만 맞추려는 경향이 있다.

HTML 기본 구조

Next.js에서는 `app/layout.tsx`가 자동으로 처리하지만, AI 출력 검증을 위해 알아둔다:

```html
<!DOCTYPE html>
<html lang="ko">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>블로그</title>
  </head>
  <body>
    <!-- 페이지 내용 -->
  </body>
</html>
```

🤖 실습: 블로그 시맨틱 뼈대 만들기

Copilot Chat(Agent 모드)에서 실행:

> "app/page.tsx를 블로그 메인 페이지로 수정해줘. 구조: header(사이트 제목 + nav) → main(게시글 카드 3개) → footer. 시맨틱 태그 사용: header, nav, main, article, footer. 게시글은 제목, 내용 미리보기, 작성자, 날짜 포함. 스타일링은 아직 하지 말고 구조만 잡아줘."

검증:

- ☐ `<header>`, `<nav>`, `<main>`, `<article>`, `<footer>` 모두 있는가?
- ☐ `<h1>` → `<h2>` → `<h3>` 계층이 올바른가?
- ☐ `className`을 사용했는가? (`class` 아님)
- ☐ `<div>` 범벅이 아닌가?

Copilot 결과가 애매하면 최소한 아래 구조는 직접 확인하며 고친다:

```
export default function Home() {
  return (
    <>
      <header>{/* 사이트 제목 + nav */}</header>
      <main>
        <section>
          <h2>최신 게시글</h2>
          <article>{/* 게시글 1 */}</article>
          <article>{/* 게시글 2 */}</article>
          <article>{/* 게시글 3 */}</article>
        </section>
      </main>
      <footer>{/* 바닥글 */}</footer>
    </>
  );
}
```

실습이 끝나면 화면에서 확인:

- ☐ 헤더, 본문, 푸터가 구분되어 보이는가?
- ☐ 게시글 카드가 3개 보이는가?
- ☐ 에러 메시지 없이 페이지가 렌더링되는가?

3.2 Tailwind CSS 기초

유틸리티 클래스란

Tailwind는 "이미 정의된 유틸리티 클래스를 HTML에 직접 붙이는" 방식이다. 별도 CSS 파일이 필요 없다.

```
{/* 전통 CSS: 별도 파일 필요 */}
<div className="card">제목</div>

{/* Tailwind: 한 파일에서 완결 */}
<div className="p-4 bg-white rounded-lg">제목</div>
```

AI와 궁합이 좋다 — Copilot이 한 파일만 보고도 전체 디자인을 파악할 수 있다.

클래스 읽는 법: `속성-값` 패턴

클래스	읽는 법	CSS 변환
`p-4`	padding 4단위	`padding: 16px`
`px-6`	padding 좌우 6단위	`padding-left: 24px; padding-right: 24px`
`mt-2`	margin 위 2단위	`margin-top: 8px`
`text-lg`	텍스트 크게	`font-size: 18px`
`font-bold`	글씨 굵게	`font-weight: 700`
`bg-blue-500`	배경 파란색 500	`background: #3b82f6`
`rounded-lg`	둥근 모서리	`border-radius: 8px`
`w-full`	가로 100%	`width: 100%`

간격: 1단위 = 4px → `p-1`=4px, `p-2`=8px, `p-4`=16px, `p-8`=32px

색상: `{색상}-{단계}` (50=밝음, 500=중간, 900=어두움)

> VS Code에서 **Tailwind CSS IntelliSense** 확장을 설치하면 클래스명 자동완성과 미리보기가 지원된다.

🤖 실습: 블로그 카드 스타일링

앞서 만든 시맨틱 뼈대에 Tailwind 스타일을 입힌다. Copilot Chat(Agent 모드):

> "현재 app/page.tsx의 블로그 레이아웃에 Tailwind CSS 스타일을 추가해줘. 게시글 카드: bg-white rounded-lg shadow p-6, hover:shadow-lg transition. 전체 레이아웃: max-w-4xl mx-auto p-4. 제목: text-lg font-bold, 본문: text-gray-600, 날짜: text-sm text-gray-400."

검증:

- ☐ `className`으로 되어 있는가?
- ☐ Tailwind 클래스가 읽히는가? (속성-값 패턴으로 해석해본다)
- ☐ 카드 배경, 여백, 그림자 차이가 눈에 보이는가?

Copilot이 너무 많은 클래스를 붙이거나 이상한 색을 넣으면, 아래 수준으로 단순화해서 다시 맞춘다:

```
<main className="max-w-4xl mx-auto p-4">
  <article className="bg-white rounded-lg shadow p-6">
    <h3 className="text-lg font-bold">React 19 새 기능</h3>
    <p className="text-gray-600">게시글 미리보기</p>
    <p className="text-sm text-gray-400">2026-03-30</p>
  </article>
</main>
```

3.3 레이아웃과 반응형

Flexbox — 한 줄 배치

내비게이션 바가 대표적이다.

그림으로 보면:

```
부모 요소: flex

┌──────────────────────────────────┐
│ [로고]        [메뉴1][메뉴2][로그인] │
└──────────────────────────────────┘
```

즉, Flexbox는 요소들을 "한 줄 방향"으로 정렬할 때 유용하다.

조금 더 단순하게 보면:

```
기본 가로 배치

┌──────────────────────┐
│ [A] [B] [C]          │
└──────────────────────┘

세로 배치 (`flex-col`)

┌──────────┐
│ [A]      │
│ [B]      │
│ [C]      │
└──────────┘
```

학생용 한 줄 정리:

- Flexbox는 메뉴바, 버튼 묶음처럼 "한 방향으로 나열"할 때 쓴다

클래스	역할
`flex`	Flexbox 활성화
`flex-col`	세로 방향 배치
`justify-between`	양 끝 정렬
`items-center`	세로축 가운데 정렬
`gap-4`	요소 사이 간격 16px

Grid — 격자 배치

게시글 카드 목록이 대표적이다.

그림으로 보면:

```
부모 요소: grid

┌──────────────────────────────┐
│ [카드1]   [카드2]            │
│ [카드3]   [카드4]            │
└──────────────────────────────┘
```

즉, Grid는 요소들을 "행과 열"로 배치할 때 유용하다.

예를 들어 게시글 카드가 여러 개라면:

```
2열 grid

┌──────────────────────────────┐
│ [게시글1]   [게시글2]        │
│ [게시글3]   [게시글4]        │
│ [게시글5]   [게시글6]        │
└──────────────────────────────┘
```

학생용 한 줄 정리:

- Grid는 카드 목록, 갤러리처럼 "칸을 나눠 배치"할 때 쓴다

클래스	역할
`grid`	Grid 활성화
`grid-cols-2`	2열 격자
`grid-cols-3`	3열 격자
`gap-4`	격자 사이 간격

반응형 브레이크포인트

Tailwind는 **모바일 우선** — 기본이 모바일, 화면이 커지면 스타일을 추가한다.

접두어	최소 너비	대상
(없음)	0px	모바일 (기본)
`sm:`	640px	큰 모바일
`md:`	768px	태블릿
`lg:`	1024px	데스크톱

예시 — 모바일 1열, 태블릿 2열:

```
<div className="grid grid-cols-1 md:grid-cols-2 gap-4">
  {/* 카드들 */}
</div>
```

🤖 실습: 반응형 레이아웃 적용

Copilot Chat(Agent 모드):

> "app/page.tsx의 블로그 레이아웃을 반응형으로 수정해줘. 내비게이션: flex justify-between items-center. 게시글 카드 목록: grid grid-cols-1 md:grid-cols-2 gap-6. footer: 중앙 정렬, text-sm text-gray-400."

검증:

- ☐ DevTools 디바이스 모드(`Cmd+Shift+M`)로 모바일/태블릿/데스크톱 확인
- ☐ 모바일에서 1열, 태블릿 이상에서 2열인가?
- ☐ 내비게이션이 가로 정렬되어 있는가?

반응형이 잘 안 되면 아래 두 줄을 먼저 확인한다:

```
<nav className="flex justify-between items-center">
<section className="grid grid-cols-1 md:grid-cols-2 gap-6">
```

실습이 끝나면 스스로 설명해본다:

1. 왜 기본값이 `grid-cols-1`인가?
2. 왜 `md:grid-cols-2`를 붙이면 태블릿부터 2열이 되는가?

흔한 AI 실수

AI 실수	올바른 코드	원인
`<div>` 범벅 (시맨틱 태그 미사용)	`<header>`, `<main>`, `<article>`	구조보다 스타일에 집중
`class="..."`	`className="..."`	HTML 학습 데이터
`for="..."`	`htmlFor="..."`	HTML 학습 데이터
h1 → h3 (h2 건너뜀)	h1 → h2 → h3	시각적 크기만 맞추려 함
반응형 미적용 (고정 너비)	`grid-cols-1 md:grid-cols-2`	데스크톱 기준으로 생성

🤖 배포

Copilot Chat(Agent 모드):

> "터미널에서 git add, commit, push를 실행해줘. 커밋 메시지: 'Ch3: 블로그 레이아웃 + Tailwind 스타일링'"

배포 전 체크:

- ☐ 개발 서버 화면에서 레이아웃이 정상인가?
- ☐ 모바일/데스크톱에서 카드 배치가 달라지는가?
- ☐ 콘솔 에러가 없는가?

push 후 확인:

1. 배포된 URL을 연다.
2. 모바일 크기와 데스크톱 크기에서 각각 확인한다.
3. 아래 3가지를 점검한다.

- ☐ 시맨틱 구조가 유지되는가?
- ☐ 카드 스타일이 적용되어 있는가?
- ☐ 반응형 1열/2열 전환이 보이는가?

핵심 정리

1. **시맨틱 태그**로 페이지 구조를 명확히 한다 (header, nav, main, article, footer)
2. **Tailwind 클래스**는 `속성-값` 패턴으로 읽는다 (p-4 = padding 16px)
3. **반응형**은 모바일 우선 — `md:`, `lg:` 접두어로 큰 화면 스타일을 추가한다

제출 안내 (Google Classroom)

```
① 배포 URL
   예: https://내프로젝트.vercel.app

② AI가 틀린 부분 1개
   예: "Copilot이 모든 태그를 <div>로 생성해서
       <header>, <main>, <article>로 수정했다."
```

스스로 점검

이 장의 실습을 마쳤다면 아래 질문에 답할 수 있어야 한다.

1. 왜 `<div>`보다 `<header>`, `<main>`, `<article>`이 더 좋은가?
2. `p-4`, `text-lg`, `md:grid-cols-2`를 각각 어떻게 읽는가?
3. Copilot이 `class` 또는 `for`를 쓰면 왜 수정해야 하는가?